

On-Premises to AWS Migration

End-to-End Migration Strategy and Design

Technical Design Document

Document Version: 1.0

Date: June 2026

Table of Contents

1. Introduction and Scope
2. AWS Migration Framework (The 7 Rs)
3. Pre-Migration Assessment and Discovery
4. Landing Zone and Account Strategy
5. Networking and Connectivity Design
6. Compute Migration (includes Step-by-Step VM Import/Export Walkthrough)
7. Storage Migration
8. Database Migration
9. Identity and Security Migration
10. Step-by-Step Migration Workflow
11. Validation, Cutover, and Rollback
12. Best Practices and Common Pitfalls

1. Introduction and Scope

1.1 Purpose

This document defines the end-to-end strategy, architecture, and methodology for migrating on-premises infrastructure — compute, storage, databases, networking, and identity — to Amazon Web Services (AWS). It serves as the master strategy document, complementing the detailed CLI and Console step-by-step runbooks for server and database migration.

1.2 Scope

- Physical and virtual servers (Windows and Linux)
- File-based and block storage systems
- Relational and non-relational databases
- On-premises networking, DNS, and identity infrastructure
- Application dependencies and integration points

2. AWS Migration Framework (The 7 Rs)

AWS defines seven common migration strategies, often referred to as the '7 Rs.' Each workload should be assessed individually to determine the most appropriate strategy.

Strategy	Description	Best Fit
Rehost	Lift-and-shift; move as-is to EC2/equivalent infrastructure	Time-constrained migrations, legacy apps
Replatform	Minor optimizations (e.g., move to RDS) without changing core architecture	Quick operational wins with low risk
Repurchase	Replace with a SaaS or AWS Marketplace alternative	Commodity functionality (CRM, ticketing, monitoring)
Refactor / Re-architect	Redesign using cloud-native services (serverless, microservices)	Strategic, long-term applications
Relocate	Move VMware-based VMs to AWS using VMware Cloud on AWS without conversion	Large VMware estates needing minimal change
Retain	Keep the workload on-premises (no migration)	Workloads with regulatory, latency, or technical constraints
Retire	Decommission the workload entirely	Redundant, unused, or obsolete systems

3. Pre-Migration Assessment and Discovery

3.1 Discovery Tools

- AWS Application Discovery Service: Agent-based or agentless discovery of on-prem servers, utilization, and network dependencies
- AWS Migration Evaluator: Builds a data-driven business case comparing current on-prem cost to projected AWS cost
- AWS Migration Hub: Centralized tracking of migration status across tools and workloads

3.2 Discovery Checklist

- Complete inventory of servers: OS, CPU, memory, disk, utilization patterns over time
- Application dependency mapping (which servers talk to which, on what ports)
- Database inventory: engine, version, size, licensing model
- Network topology: VLANs, firewall rules, load balancers, DNS records
- Identity infrastructure: Active Directory domains, trust relationships, group policies
- Compliance and data classification requirements per workload

3.3 Workload Prioritization (The 7 Rs Applied)

13. Categorize each discovered workload into one of the 7 R strategies
14. Group workloads into migration waves based on dependency relationships (move dependent systems together)
15. Prioritize low-risk, low-complexity workloads for early waves to build organizational confidence
16. Schedule high-risk or highly-dependent workloads for later waves once the process is proven

4. Landing Zone and Account Strategy

4.1 Multi-Account Strategy using AWS Control Tower

AWS recommends a multi-account strategy to isolate workloads, environments, and billing. A typical landing zone structure includes:

- Management Account: Root account for billing and AWS Organizations management
- Security/Audit Account: Centralized logging (CloudTrail, Config) and security tooling
- Shared Services Account: Centralized AD, DNS, and shared networking resources
- Production Account(s): Isolated production workloads, segmented by business unit or application
- Non-Production Account(s): Development, testing, and staging environments

4.2 Setting Up the Landing Zone (CLI)

```
# Enable AWS Organizations aws organizations create-organization --feature-set ALL # Create a new account under the organization aws organizations create-account \ --email production@company.com \ --account-name "Production-Workloads" # Enable AWS Control Tower (typically via Console, then managed via CLI/API afterward) aws controltower create-landing-zone \ --manifest file://landing-zone-manifest.json \ --version "3.3"
```

4.3 Tagging Strategy

Define a consistent resource tagging strategy from day one for cost allocation, automation, and governance:

Tag Key	Example Value	Purpose
Environment	production / staging / dev	Environment-based cost allocation and access control
Application	order-management-system	Group resources by application for cost and ownership tracking
Owner	platform-team@company.com	Accountability and contact for the resource
MigrationWave	wave-1 / wave-2	Track migration progress and rollback grouping
CostCenter	CC-1024	Financial chargeback and reporting

5. Networking and Connectivity Design

5.1 Establishing Hybrid Connectivity

Option 1: Site-to-Site VPN

```
# Create a Customer Gateway representing the on-prem VPN device aws ec2 create-customer-gateway \ --type ipsec.1 \ --public-ip 203.0.113.10 \ --bgp-asn 65000 # Create a Virtual Private Gateway and attach to VPC aws ec2 create-vpn-gateway --type ipsec.1 aws ec2 attach-vpn-gateway \ --vpc-id vpc-12345678 \ --vpn-gateway-id vgw-12345678 # Create the VPN connection aws ec2 create-vpn-connection \ --type ipsec.1 \ --customer-gateway-id cgw-12345678 \ --vpn-gateway-id vgw-12345678 \ --options StaticRoutesOnly=true
```

Option 2: AWS Direct Connect (for sustained, high-bandwidth migration)

- Order a Direct Connect connection through the AWS Console or an AWS Direct Connect Partner
- Provision a Virtual Interface (VIF) — private VIF for VPC access, public VIF for AWS public services
- Configure BGP routing between on-prem router and the Direct Connect location
- Recommended for migrations exceeding several TB or requiring consistent low-latency connectivity

5.2 VPC Design

```
# Create VPC aws ec2 create-vpc --cidr-block 10.0.0.0/16 # Create subnets across multiple AZs aws ec2 create-subnet --vpc-id vpc-12345678 --cidr-block 10.0.1.0/24 --availability-zone us-east-1a aws ec2 create-subnet --vpc-id vpc-12345678 --cidr-block 10.0.2.0/24 --availability-zone us-east-1b # Create and attach Internet Gateway (for public subnets) aws ec2 create-internet-gateway aws ec2 attach-internet-gateway --vpc-id vpc-12345678 --internet-gateway-id igw-12345678
```

5.3 DNS Strategy

- Use Amazon Route 53 Resolver with conditional forwarding rules to maintain hybrid DNS resolution during migration
- Configure inbound/outbound resolver endpoints to allow on-prem DNS to resolve AWS-hosted resources and vice versa
- Plan final DNS cutover (public records) only after full validation is complete

6. Compute Migration

6.1 AWS Application Migration Service (MGN) — Recommended Approach

AWS MGN is the recommended lift-and-shift tool for server migration, using continuous block-level replication to minimize cutover downtime to minutes.

```
# Step 1: Install the AWS Replication Agent on the source server # (Linux) wget -O ./aws-replication-installer-init.py https://aws-application-migration-service-us-east-1.s3.us-east-1.amazonaws.com/latest/linux/aws-replication-installer-init.py sudo python3 aws-replication-installer-init.py \ --region us-east-1 \ --aws-access-key-id <ACCESS_KEY> \ --aws-secret-access-key <SECRET_KEY> # Step 2: Monitor replication status aws mgn describe-source-servers --region us-east-1 # Step 3: Launch a test instance once replication reaches 100% aws mgn start-test \ --source-server-id s-1234567890abcdef0 # Step 4: After validation, launch the cutover instance aws mgn start-cutover \ --source-server-id s-1234567890abcdef0
```

6.2 Manual EC2 Provisioning (Alternative for Simple Workloads)

```
aws ec2 run-instances \ --image-id ami-0c55b159cbfafa1f0 \ --instance-type m5.2xlarge \ --key-name migration-key \ --security-group-ids sg-12345678 \ --subnet-id subnet-12345678 \ --block-device-mappings DeviceName=/dev/sda1,Ebs={VolumeSize=100,VolumeType=gp3}
```

6.3 Right-Sizing Guidance

AWS MGN and Migration Hub provide right-sizing recommendations based on observed on-prem utilization. Avoid simply matching on-prem core/RAM counts 1:1 — cloud instance performance characteristics differ, and right-sizing based on actual utilization typically reduces cost by 20-40%.

6.4 Walkthrough: Manual VM Import/Export (Step-by-Step)

AWS MGN (Section 6.1) is the recommended approach for most production migrations because it replicates continuously and minimizes downtime. However, for smaller workloads, lab environments, or one-off migrations, a simpler manual path exists: export the VM from your hypervisor as an OVA file, upload it to S3, and import it directly as a new EC2 instance using the AWS CLI VM Import/Export feature. The steps below walk through this manual path in plain language, exactly as you would click through it.

Step 1 — Power off the source VM

Before exporting, the virtual machine must be powered off (not just paused or saved). In your hypervisor manager (VirtualBox, VMware, or Hyper-V), right-click the VM, choose to close it, and select "Power off the machine."

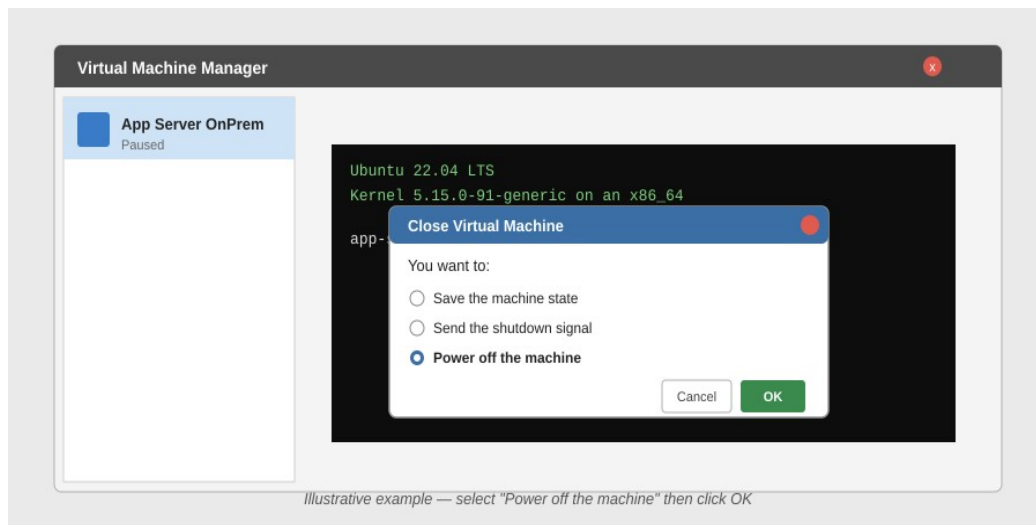


Figure 6.1 — Powering off the source VM before export

Step 2 — Export the VM to OVA format

Right-click the powered-off VM and choose the export option. Select the Open Virtualization Format (.ova) — this packages the disk image and VM configuration into a single portable file that AWS can later read.

Figure 6.2 — Exporting the VM to an .ova file

This export can take anywhere from a few minutes to over an hour depending on the size of the virtual disk. Once complete, you will have a single .ova file on your local machine (for example, AppServer-OnPrem.ova).

Step 3 — Create an S3 bucket and upload the OVA file

In the AWS region where you want the server to live, create an S3 bucket and upload the exported .ova file to it. This bucket acts as the staging area AWS reads from during import.

Create the bucket (CLI) `aws s3 mb s3://migration-bucket-2026 --region us-east-1` # Upload the exported OVA file `aws s3 cp AppServer-OnPrem.ova s3://migration-bucket-2026/`

Name	Type	Size	Uploaded on
AppServer-OnPrem.ova	ova file	8.2 GB	June 28, 2026
trust-policy.json	json file	312 B	June 28, 2026

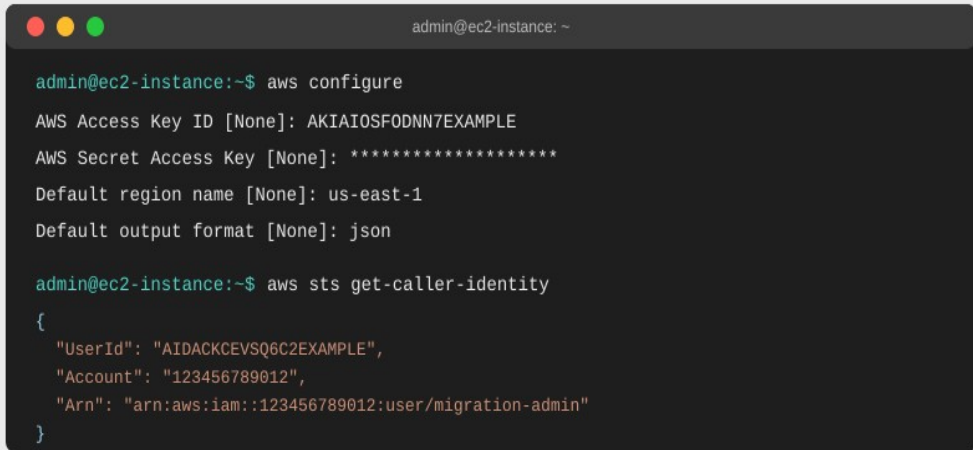
Upload succeeded
AppServer-OnPrem.ova was uploaded to migration-bucket-2026

Illustrative example — exported .ova file uploaded to an S3 bucket

Figure 6.3 — The .ova file after a successful upload to S3

Step 4 — Install and configure the AWS CLI

Install the AWS CLI on your laptop, or simply launch a small Amazon Linux EC2 instance, which comes with the AWS CLI already installed. Either way, run `aws configure` and enter the access key, secret key, region, and output format for an IAM user with EC2, S3, and IAM permissions.



```
admin@ec2-instance: ~  
  
admin@ec2-instance:~$ aws configure  
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE  
AWS Secret Access Key [None]: *****  
Default region name [None]: us-east-1  
Default output format [None]: json  
  
admin@ec2-instance:~$ aws sts get-caller-identity  
{  
  "UserId": "AIDACKCEVSQ6C2EXAMPLE",  
  "Account": "123456789012",  
  "Arn": "arn:aws:iam::123456789012:user/migration-admin"  
}
```

Illustrative example — values shown are sample placeholders, not real credentials

Figure 6.4 — Configuring the AWS CLI and verifying the identity

Step 5 — Create the vmimport IAM role

AWS's import service needs permission to read your S3 bucket and write the resulting image. This is done once, by creating a role named `vmimport` with a trust policy and a permissions policy.

```
# trust-policy.json { "Version": "2012-10-17", "Statement": [ { "Effect": "Allow",  
"Principal": { "Service": "vmie.amazonaws.com" }, "Action": "sts:AssumeRole",  
"Condition": { "StringEquals": { "sts:Externalid": "vmimport" } } ] } # Create the role  
aws iam create-role \ --role-name vmimport \ --assume-role-policy-document "file://trust-  
policy.json"
```

```
# role-policy.json — grants S3 read and EC2 image permissions { "Version": "2012-10-17",  
"Statement": [ { "Effect": "Allow", "Action": [ "s3:GetBucketLocation",  
"s3:GetObject", "s3:ListBucket", "s3:PutObject", "s3:GetBucketAcl" ], "Resource": [  
"arn:aws:s3::migration-bucket-2026", "arn:aws:s3::migration-bucket-2026/*" ] }, {  
"Effect": "Allow", "Action": [ "ec2:ModifySnapshotAttribute", "ec2:CopySnapshot",  
"ec2:RegisterImage", "ec2:Describe*" ], "Resource": "*" } ] } # Attach the policy to  
the role  
aws iam put-role-policy \ --role-name vmimport \ --policy-name vmimport \ --policy-  
document "file://role-policy.json"
```

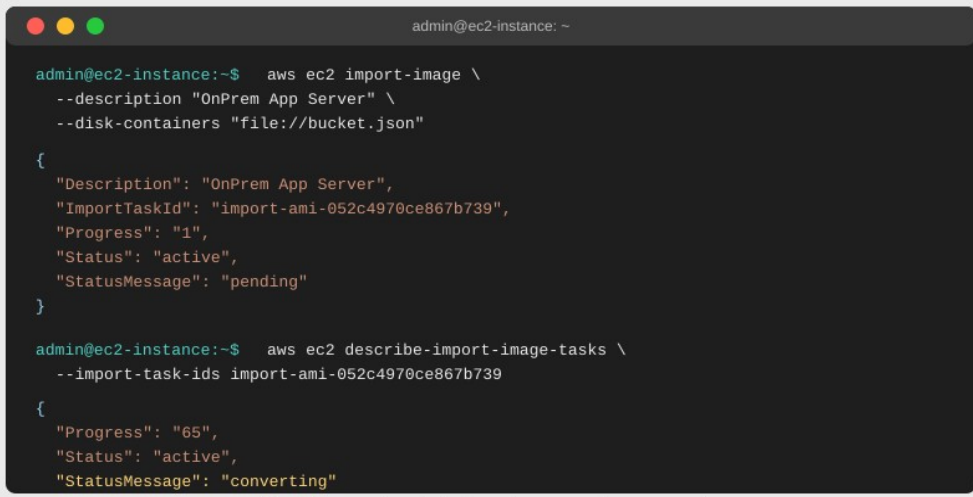
Step 6 — Start the import

Create a small file telling AWS which bucket and object to import, then kick off the import task.

```
# bucket.json [ { "Description": "My Server OVA", "Format": "ova", "UserBucket":  
{ "S3Bucket": "migration-bucket-2026", "S3Key": "AppServer-OnPrem.ova" } } ] #  
Start the import aws ec2 import-image \ --description "OnPrem App Server" \ --disk-  
containers "file://bucket.json"
```

Step 7 — Check the import progress

The import runs in the background and typically takes 20 minutes to a few hours depending on disk size. Check on it any time with `describe-import-image-tasks`.



```
admin@ec2-instance: ~  
  
admin@ec2-instance:~$ aws ec2 import-image \  
--description "OnPrem App Server" \  
--disk-containers "file://bucket.json"  
  
{  
  "Description": "OnPrem App Server",  
  "ImportTaskId": "import-ami-052c4970ce867b739",  
  "Progress": "1",  
  "Status": "active",  
  "StatusMessage": "pending"  
}  
  
admin@ec2-instance:~$ aws ec2 describe-import-image-tasks \  
--import-task-ids import-ami-052c4970ce867b739  
  
{  
  "Progress": "65",  
  "Status": "active",  
  "StatusMessage": "converting"  
}
```

Illustrative example — task IDs and progress values are samples

Figure 6.5 — Starting the import and checking progress

The `StatusMessage` field moves through stages such as `pending`, `converting`, and finally `completed`. Once complete, AWS automatically registers a new AMI (Amazon Machine Image) from your on-premises VM.

Step 8 — Launch the imported server

Once the import finishes, the new AMI appears under `EC2 > Images > AMIs`. Launch an instance from it the same way you would launch any other EC2 instance. Your on-premises server is now running natively in AWS.

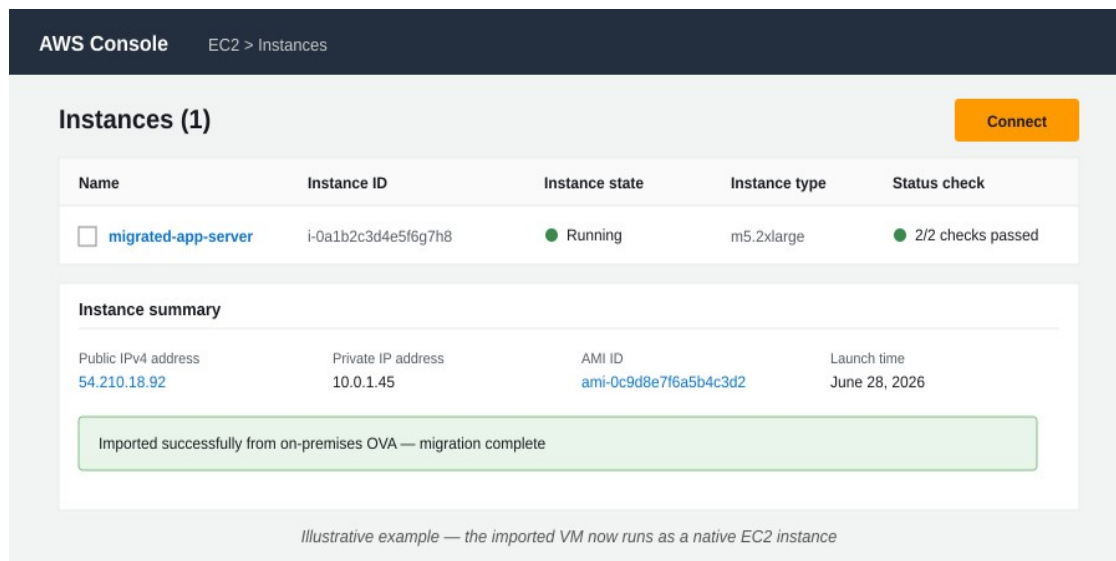


Figure 6.6 — The migrated server running as a native EC2 instance

Quick reference: the full command sequence

Step	What it does
aws s3 mb / aws s3 cp	Create the staging bucket and upload the .ova file
aws configure	Set up CLI credentials and default region
aws iam create-role --role-name vmimport	Create the role AWS uses to perform the import
aws iam put-role-policy	Grant that role access to your S3 bucket and EC2 image actions
aws ec2 import-image	Start converting the .ova file into an AMI
aws ec2 describe-import-image-tasks	Check progress until status shows completed
aws ec2 run-instances (or launch via Console)	Boot a real EC2 instance from the imported AMI

Try it yourself

A good way to build confidence with this process is a small hands-on exercise:

17. Create a VM in VirtualBox, VMware, or Hyper-V running Ubuntu
18. Install a simple web application and a MySQL database on it
19. Follow Steps 1-8 above to migrate that VM into AWS as a real EC2 instance

7. Storage Migration

7.1 AWS DataSync (File-Based Transfer)

```
# Create source location (on-prem NFS share via DataSync agent) aws datasync create-  
location-nfs \ --subdirectory "/data" \ --server-hostname 10.0.1.50 \ --on-prem-config  
AgentArns=arn:aws:datasync:us-east-1:123456789012:agent/agent-12345678 # Create  
destination location (S3 or EFS) aws datasync create-location-s3 \ --s3-bucket-arn  
arn:aws:s3::migration-bucket \ --subdirectory /server-data \ --s3-config  
BucketAccessRoleArn=arn:aws:iam::123456789012:role/DataSyncS3Role # Create and start  
the transfer task aws datasync create-task \ --source-location-arn <source-arn> \ --  
destination-location-arn <dest-arn> \ --name "Storage-Migration-Task" aws datasync start-  
task-execution --task-arn <task-arn>
```

7.2 AWS Snow Family (Large-Scale Offline Transfer)

Service	Use Case	Capacity
AWS Snowcone	Small edge/offline transfers	Up to 8 TB
AWS Snowball Edge (Storage Optimized)	Mid-to-large scale data transfer with local compute	Up to 80 TB
AWS Snowmobile	Exabyte-scale data center migrations	Up to 100 PB per Snowmobile

7.3 Choosing the Target Storage Service

On-Prem Source	Recommended AWS Target	Notes
File server (SMB/NFS)	Amazon FSx or Amazon EFS	FSx for Windows File Server (SMB) or EFS (NFS, Linux)
Block storage (SAN)	Amazon EBS	Attach to EC2; size and IOPS configurable per volume
Object/archive storage	Amazon S3 (with lifecycle policies to S3 Glacier)	Use storage classes to optimize cost by access frequency
Backup/archive tapes	AWS Snowball + S3 Glacier Deep Archive	For long-term retention migration

8. Database Migration

Detailed database migration strategy, tools, and best practices are covered comprehensively in the companion document: Database Migration – Strategies, Tools, and Best Practices. This section summarizes the AWS-specific service mapping.

8.1 AWS Database Service Mapping

On-Prem Database	AWS Target Recommendation	Migration Tool
MySQL / MariaDB	Amazon RDS for MySQL/MariaDB or Aurora MySQL	AWS DMS, mysqldump
PostgreSQL	Amazon RDS for PostgreSQL or Aurora PostgreSQL	AWS DMS, pg_dump/pg_restore
SQL Server	Amazon RDS for SQL Server	AWS DMS, Native Backup/Restore
Oracle	Amazon RDS for Oracle or migrate to Aurora PostgreSQL (re-platform)	AWS DMS + AWS SCT, Data Pump
MongoDB	Amazon DocumentDB (MongoDB-compatible)	AWS DMS, mongodump/mongorestore
Self-managed NoSQL/key-value	Amazon DynamoDB	AWS DMS (where supported), custom ETL

8.2 Quick Reference: DMS Migration Command Sequence

```
# 1. Create replication instance aws dms create-replication-instance \ --replication-instance-identifier onprem-to-aws-migration \ --replication-instance-class dms.c5.2xlarge \ --allocated-storage 500 # 2. Create source (on-prem) and target (RDS) endpoints aws dms create-endpoint --endpoint-identifier source-onprem-db --endpoint-type source --engine-name mysql ... aws dms create-endpoint --endpoint-identifier target-rds-db --endpoint-type target --engine-name mysql ... # 3. Create and start the replication task aws dms create-replication-task \ --replication-task-identifier onprem-migration-task \ --migration-type full-load-and-cdc \ --table-mappings file://table-mappings.json aws dms start-replication-task \ --replication-task-arn <task-arn> \ --start-replication-task-type start-replication
```

Refer to the Database Migration companion document for complete schema conversion, validation, and cutover procedures.

9. Identity and Security Migration

9.1 Active Directory Integration

- Deploy AWS Directory Service (AWS Managed Microsoft AD) to extend or replace on-prem Active Directory
- Alternatively, establish a trust relationship between on-prem AD and AWS Managed Microsoft AD for a hybrid identity model
- Join migrated EC2 instances to the appropriate domain for consistent group policy and authentication

9.2 IAM Strategy

- Design IAM roles and policies following least-privilege principles — avoid replicating broad on-prem admin access models directly
- Use IAM Identity Center (formerly AWS SSO) integrated with on-prem identity provider (AD, Okta, Azure AD) for centralized federated access
- Enable AWS CloudTrail across all accounts for full audit logging from day one

9.3 Security Baseline

- Enable AWS Config for continuous compliance monitoring against organizational policies
- Enable Amazon GuardDuty for threat detection across all migrated accounts
- Encrypt data at rest (EBS, S3, RDS) using AWS KMS, and enforce TLS for data in transit
- Replicate or improve upon existing on-prem network segmentation using Security Groups and NACLs

10. Step-by-Step Migration Workflow

20. Phase 1 — Discovery and Assessment: Run AWS Application Discovery Service and Migration Evaluator (Section 3)
21. Phase 2 — Landing Zone Setup: Establish AWS Organizations, Control Tower, and account structure (Section 4)
22. Phase 3 — Connectivity: Establish VPN or Direct Connect and configure hybrid DNS (Section 5)
23. Phase 4 — Identity Foundation: Deploy AWS Managed Microsoft AD and IAM Identity Center (Section 9)
24. Phase 5 — Pilot Migration Wave: Migrate a small, low-risk workload end-to-end to validate tooling and process
25. Phase 6 — Storage Migration: Begin DataSync or Snow Family transfers for file/object data (Section 7)
26. Phase 7 — Database Migration: Execute DMS full-load and CDC migration per the companion Database Migration document (Section 8)
27. Phase 8 — Compute Migration: Use AWS MGN for continuous replication of remaining servers (Section 6)
28. Phase 9 — Validation: Execute functional, performance, and security testing in AWS
29. Phase 10 — Cutover: Execute MGN cutover and DNS/traffic redirection per wave
30. Phase 11 — Decommission: After stability confirmation, decommission on-prem infrastructure for each migrated wave

11. Validation, Cutover, and Rollback

11.1 Validation Checklist

- All MGN-replicated servers pass test-launch validation before cutover
- Database row counts and checksums validated between source and RDS target
- Application functional and performance test suites pass in the AWS environment
- Security baseline (GuardDuty, Config, CloudTrail) active and reporting correctly
- DNS resolution validated for both internal (Route 53 Resolver) and external (public Route 53) records

11.2 Cutover Execution

31. Confirm replication lag (MGN and DMS) is at or near zero
32. Freeze writes on-prem systems during the planned maintenance window
33. Execute AWS MGN cutover to launch final EC2 instances from latest replicated state
34. Stop DMS replication task and perform final validation
35. Update DNS records and load balancer configurations to route traffic to AWS
36. Monitor application and infrastructure health closely for the first 24-48 hours

11.3 Rollback Plan

37. Retain on-premises source servers and databases in a running, untouched state for a defined retention window (typically 1-2 weeks per wave)
38. Keep hybrid DNS resolution active to support fast rollback if issues are detected
39. Define explicit rollback trigger criteria and a decision owner before each wave's cutover
40. If rollback occurs, reconcile any writes made only in AWS during the failed cutover window
41. Conduct a post-incident review before rescheduling the cutover for that wave

12. Best Practices and Common Pitfalls

12.1 Best Practices

- Always run a pilot wave with non-critical workloads before migrating production-critical systems
- Use AWS MGN over manual AMI export/import — it dramatically reduces cutover downtime via continuous replication
- Right-size EC2 instances based on actual observed utilization, not a 1:1 match to on-prem specs
- Establish the landing zone (accounts, networking, identity) completely before beginning workload migration
- Group workloads into migration waves based on application dependencies, not arbitrary batches
- Automate validation (row counts, checksums, smoke tests) so it can be re-run quickly for every wave
- Maintain the hybrid connectivity and rollback window beyond the cutover date for each wave

12.2 Common Pitfalls

Pitfall	Impact	Mitigation
Migrating without dependency mapping	Broken applications post-migration due to missed dependencies	Use AWS Application Discovery Service to map dependencies before planning waves
Undersized replication/migration instances	Slow transfers, missed maintenance windows	Right-size DMS/MGN infrastructure based on data volume and change rate
No landing zone governance from day one	Security and cost sprawl as workloads scale	Establish Control Tower, tagging, and IAM guardrails before migrating workloads
Treating all workloads with the same migration strategy	Wasted effort or poor outcomes for workloads needing refactor vs. rehost	Apply the 7 Rs framework per workload during assessment
Decommissioning source too early	No rollback path if post-migration issues surface	Maintain a defined on-prem retention window per migration wave